

C Programming

Trainer : Rajiv K

Email: rajiv.kamune@sunbeaminfo.com



How To Learn Programming

- Learn the Concept in Lecture
- Understand the program flow and concept in lecture
- **“Practice the code in lab by typing the code in lab by your own”**
- Discuss with your friend and teach them.



Agenda

- C Programming Subject Contents
- C development environment
- Introduction to C
- Control flow
- Functions
- Storage class
- Pointer
- Type qualifiers
- Preprocessor Directives
- Array
- Dynamic memory allocation
- Structure & union
- Enum
- File IO
- Function pointer
- Bitwise operator and Advanced features



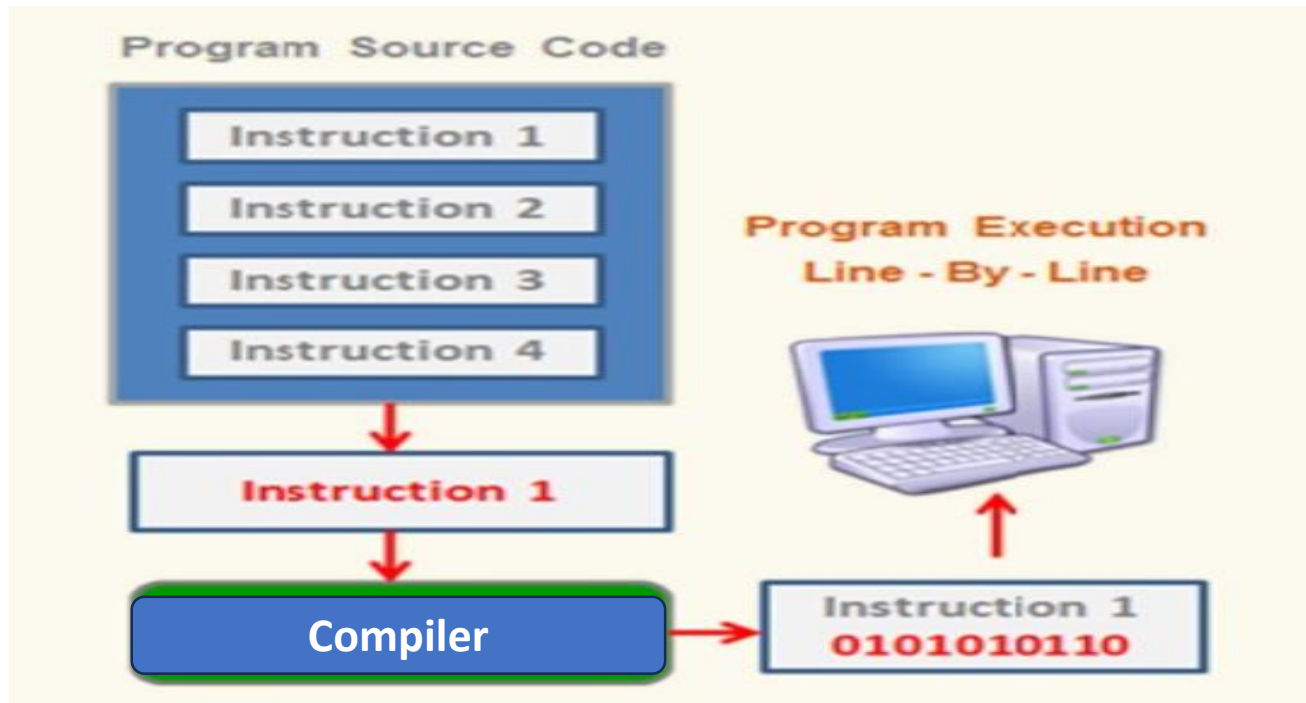
Computer and Program

What is Computer?

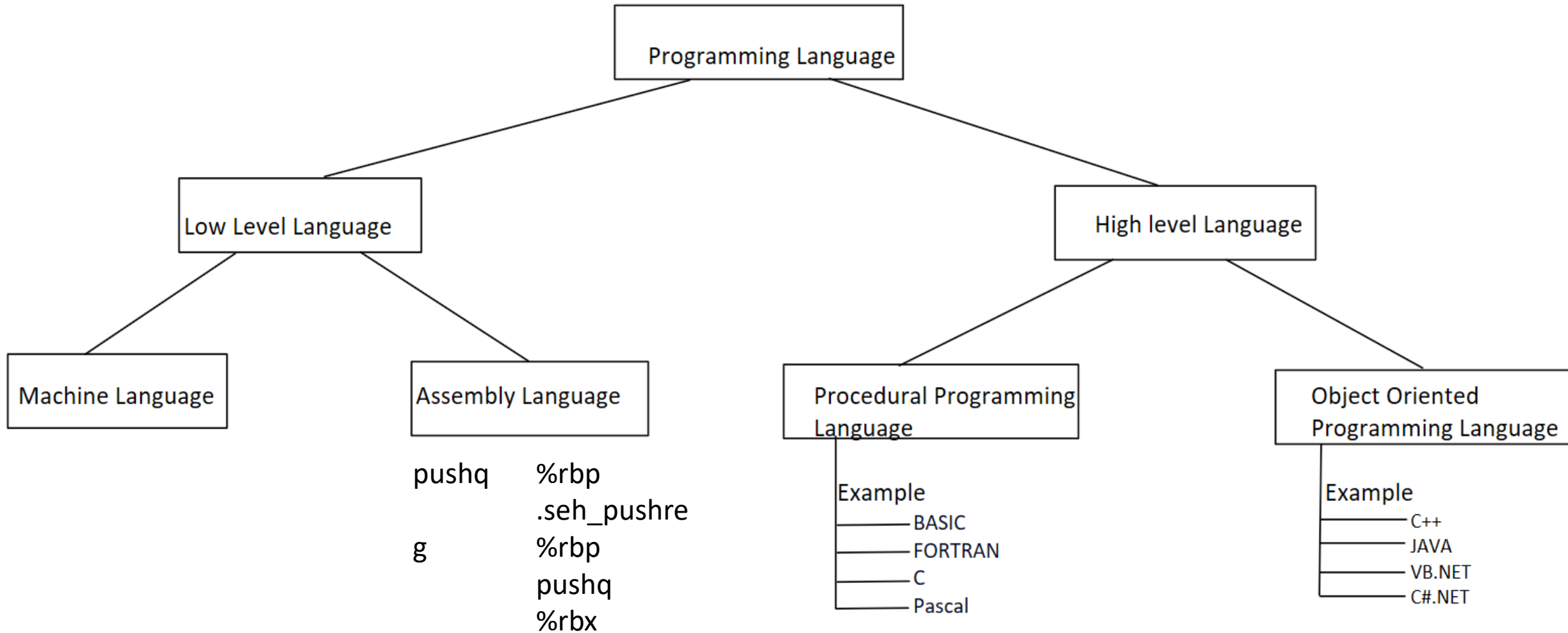
- It is a machine/hardware/digital device which does various tasks for the user efficiently and effectively.

What is Program?

- Set of instructions given to the machine to do specific task.



Classification of Languages



Classification of Languages

- **Low Level Language**

- The low level language is a programming language where machine instructions can be given **in 0 and 1 form**.

- **Machine Level Language**

- The machine-level language is a language that consists of a **set of instructions that are in the binary form 0 or 1**. As we know that computers can understand only machine instructions, which are in binary digits, i.e., 0 and 1, so the instructions given to the computer can be only in binary codes.
 - **Performance is good** as we are directly writing the program on machine
 - **Platform dependent**
 - **Difficult to program**
 - **Error prone**



Classification of Languages

- **Assembly Language**

- The assembly language contains **some human-readable instructions (consists of numbers, symbols, abbreviations)**, The language was introduced in **1952**.
- As we know that **computers can only understand the machine-level instructions**, so we require a translator that converts the assembly code into machine code. The **translator used** for translating the code is known as **an assembler**.
 - Easier to understand and use and to locate errors
 - **Platform dependent**
 - Knowledge of hardware will help to program Machine level coding
 - Execution is slower compared to machine level language.

- **High Level Language**

- Make **easy communication** to computer system
- **Independent to particular type of computer**
- More **close to human language** than machine language.
Compiler helps to convert instructions to machine understandable form.
- Takes **additional time to translate the instructions to machine** understandable instructions.



History Of C Programming

- C language was developed by **Dennis Ritchie** in **1972** at **AT & T Bell Labs** on PDP-11 (Programmed Data Processor) machine.
- It was developed while porting UNIX from PDP-7 to PDP-11.
- Many features of C are inspired from B language (Ken Thompson) and BCPL (Martin Richards).
- Initial release of C is referred as K & R C.



Standardization

- C was standardized by **ANSI in 1989**. This is referred as C89.
- Standardization ensures C code to **remain portable**.
- C standard is revised **multiple times** to **add new features** in the language.
- C89 – First ANSI standard
- C90 – ANSI standard adopted by ISO
- C99 – Added few C++ features like bool, inline, etc.
- C11 – Added multi-threading feature.
- C17 – Few technical corrections.



Compilation & Execution of C program

- **Preprocessor:-**

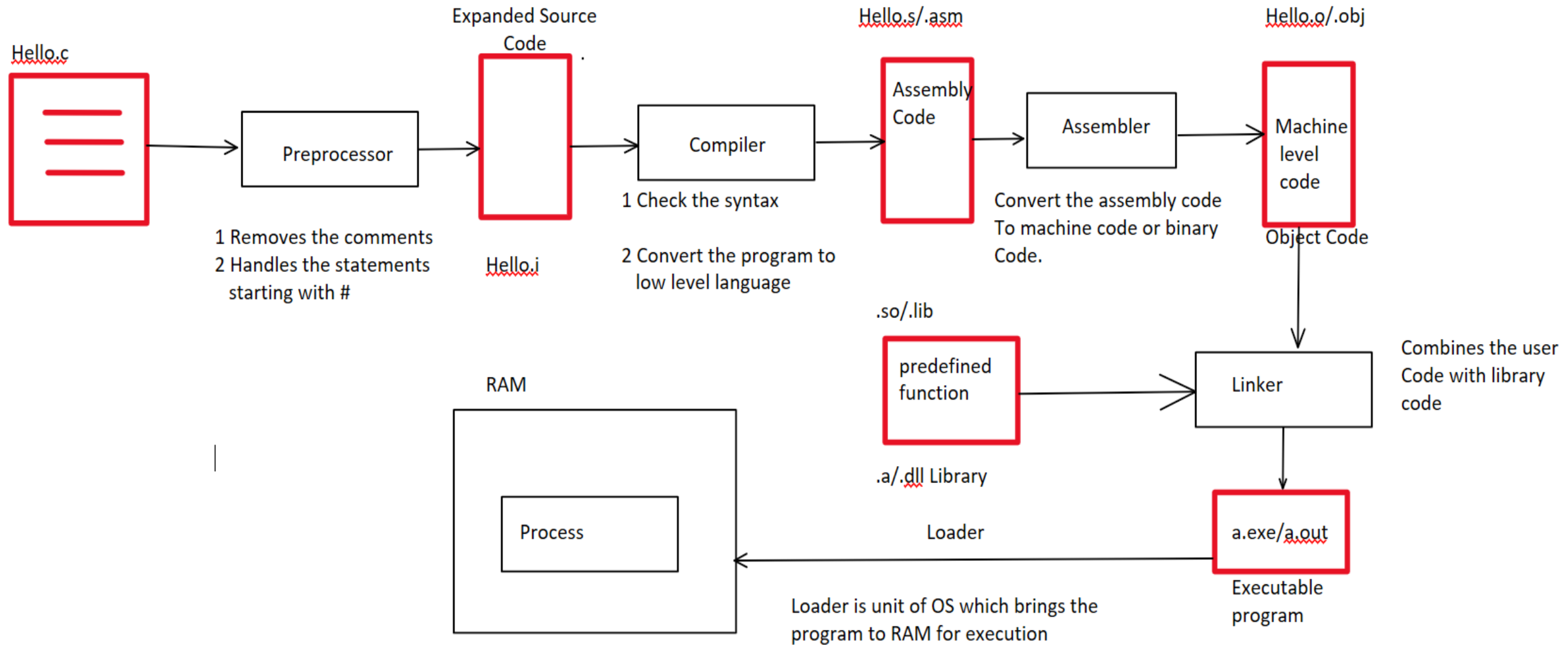
- A preprocessor accepts the inputs in source language and produces output source program that is acceptable to the compiler.
- Main task of preprocessor is to remove the comments and handle all the statements starting with #

- **Linker:-**

- Single programmer can write small program and store it in single source code file , However **Large size software consist of several thousands and several millions of code** , to take care of this approach software developers generally follow the **modular approach**.
- In modular approach software consist of multiple source file , In this case we use the software called as linker to combine all object programs and to convert into final executable.
- **Linker is a software** that takes multiple object files and fits them together to assemble into **final executable**.



Compilation & Execution of C program



Toolchain & IDE

- **Toolchain is set of tools to convert high level language program to machine level code.**
 - Preprocessor
 - Compiler
 - Assembler
 - Linker
 - Debugger
 - source code editor
 - build tools, etc.
- **Popular compiler (toolchains)**
 - GCC
 - Visual Studio
- **IDE – Integrated development environment**
 - Visual Studio
 - Eclipse
 - VS Code (+gcc), etc



Strengths of C Language

- Low-level nature **makes it very efficient** (pointers, data structures)
- Language itself is **very readable** (macros, enum, functions, ...)
- **Many compilers available** for a wide range of machines and platforms
- Standardization improves **portability of code**
- Efficient in terms of coding effort needed for a wide variety of programming applications. **Can access OS features** (functions/commands)
- **Effective memory access** (bitwise operators, bit-fields, unions)
- **Extensive library functions** (math, strings, file IO, ...)



Applications

- System programming
- OS development
- Device drivers
- System utilities
- Embedded programming
- IoT development
- Language development
- Compiler development
- Achievements (tiobe.com)
- In top-2 languages in last 40 years.
- Language of year: 2019, 2017, 2008.



Hello World

Source Code

```
// Hello World program
#include <stdio.h>

int main() {
printf("Hello World\n");
return 0;
}
```

- Commands

To compile the code

- cmd> gcc hello.c

To execute the code

- cmd> .a.exe



Hello World

- printf() – library function
- stdio.h – header file
- main() – entry point function
- void main() { ... }
- int main() { ... }
- int main(void) { ... }
- int main(int argc, char *argv[]) { ... }
- int main(int argc, char *argv[], char *envp[]) { ... }
- return 0 – exit status



Algorithm

- **Algorithm - Logical flow** mentioned for given problem statement

Algorithm can be represented in two ways:

- **1. Pseudocode** - Logical flow represented using simple english language
- **2. Flowchart** - pictorial representation of logical flow.
- Accept two numbers from user and display addition of them.

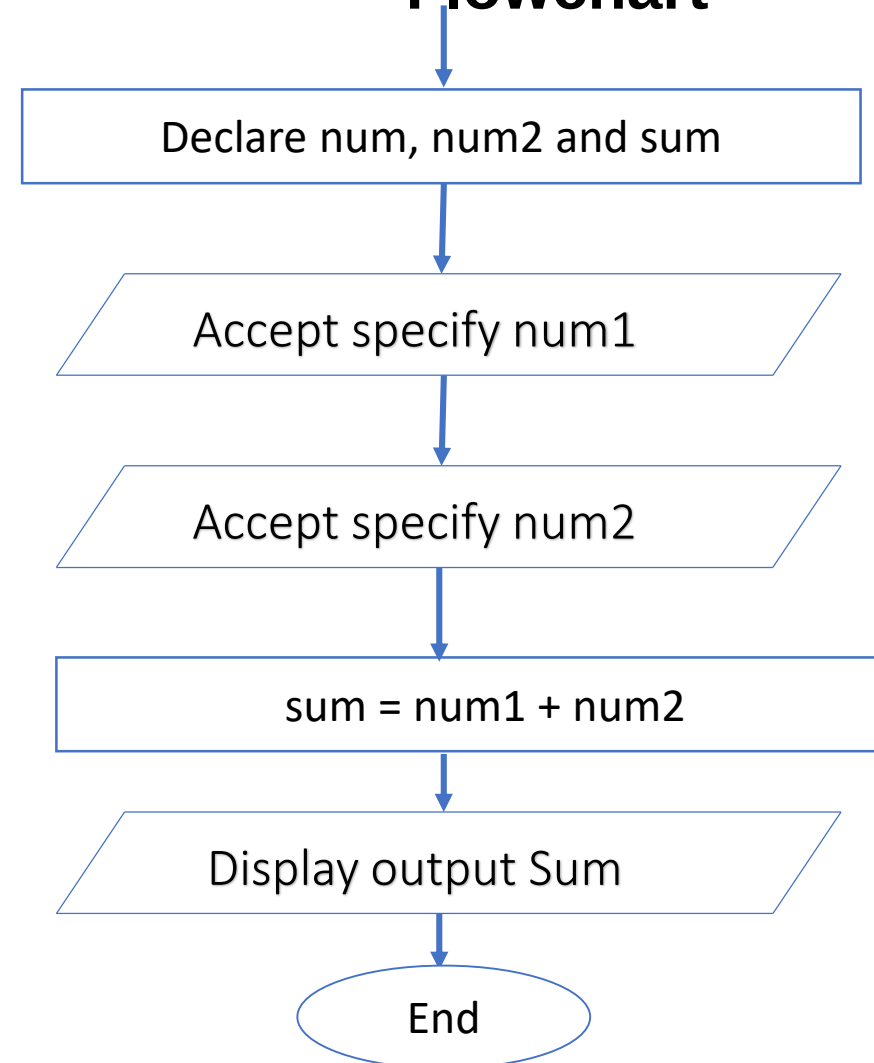


Pseudocode and Flowchart

- **Pseudocode**

- Start
- Declare num1,num2,sum
- Display "Specify number"
- Accept num1
- Display "Specify number"
- Accept num2
- $\text{sum} = \text{num1} + \text{num2}$
- Display "Sum is",sum
- end

Flowchart



Thank You

